

ICS - lab0 实验报告

一、问题回答

1.1 多人协同开发经历与分工方式

我之前有过多人协同开发的经历。在数学建模比赛中，我们小组三人共同完成一个项目。我们采用的为最原始的方式：把整个项目文件夹打包成压缩包，通过微信群传递，每个人下载后修改不同的部分，再统一由一个人手动合并。这种方式效率很低，经常出现有人没找到群里发的最新版文档，从而修改的是旧版本，导致重复工作；或由于协商好任务分配导致已经改好的部分被覆盖；或发现新改好的版本没有之前的版本好，但是由于之前的版本没备份导致无法追回；甚至出现过文件命名混乱导致版本无法追溯的情况。后来我们尝试改用腾讯文档等在线协作平台编写文档，代码部分则约定每人负责不同的模块文件，由专人负责汇总，冲突才有所减少，但仍然无法方便地查看每个人的工作历程。

1.2 Git 为什么要设计“暂存-提交”两个步骤

Git 将文件的改动分为工作区、暂存区和版本库三个层次，“暂存 - 提交”两步设计主要有以下考虑：

(1) 提交是对仓库状态的快照，必须完整、可回溯。如果直接从工作区一步提交，那么一次提交的内容就等于当前磁盘上的全部改动，无法控制本次提交包含哪些修改。而暂存区相当于一次提交的草稿，可以自由挑选要进入本次提交的改动，把无关的调试代码留在工作区。

(2) 暂存区让提交可以被精心组织。通过暂存区，开发者可以把一次大的修改拆分成多个主题清晰、大小合适的提交，每个提交只做一件事，这样提交历史更清晰，出问题时可以快速定位和撤销，也方便他人 code review。

(3) 暂存区还承担了冲突解决的中转作用。在执行 merge 出现冲突时，解决后的文件必须先 git add 到暂存区，再 git commit。本次实验中解决合并冲突就经历了这一过程。

1.3 git branch 与 git branch -a 的区别

`git branch` 列出本地仓库中的所有本地分支，当前所在分支前会标有 * 号，例如输出 * main 和 feature 表示本地有 main 与 feature 两个分支，当前在 main 分支上。

`git branch -a` 中的 -a 即 --all，表示列出所有分支，除了本地分支之外，还包括远程跟踪分支。远程跟踪分支是本地对远程仓库分支状态的只读引用，记录了上次 fetch 或 push 时远程分支指向的位置。例如 `git branch -a` 的输出可能为：* main、feature、remotes/origin/main、remotes/origin/feature，其中 origin 为远程仓库在 clone 之后的地址的默认简称。

综上所述，`git branch` 只显示本地分支；`git branch -a` 显示本地分支与远程跟踪分支的全部，常用于查看远程是否存在某一分支。

二、实验步骤

2.1 建立个人仓库并完成 main.c 中的 TODO

首先将课程提供的仓库 fork/复制为个人仓库，并克隆到本地。打开 main.c，其内容包含一个 TODO：

```
#include <stdio.h>

int main()
{
    // @TODO: print a sentence you want.
    printf("Hello, world!\n");
}
```

我完成了其中的 TODO，将 printf 语句打印的内容改为“ics-2026”。随后依次执行：

```
gcc main.c -o main.o
./main.o
git add .
git commit -m "main-task2"
```

2.2 阅读文章概括与“为什么要学习 Git”

我选择阅读了“Commit message 和 Change log 编写指南”和“语义化版本 2.0.0”两个网页。

2.2.1 Commit message 和 Change log 编写指南

网页介绍了 Commit message 的规范写法：

(1) 每次提交的信息应由标题 (header)、正文 (body) 和脚注 (footer) 三部分组成。

(2) 标题要求不超过 50 个字符，以动词开头使用一般现在时，且不加句号；如果一次提交破坏了向后兼容性，应在 footer 中以 BREAKING CHANGE 开头说明。文章还说明了如何利用这些规范的提交信息，借助工具自动生成项目的 Change log (变更日志)，从而让每一次版本变了什么、为什么变都有据可查。

2.2.2 语义化版本 2.0.0

该网页定义了版本号格式“主版本号.次版本号.修订号” (X.Y.Z)，并规定了版本号递增的规则：做了不兼容的 API 修改时递增主版本号；以向后兼容的方式新增功能时递增次版本号；只做向后兼容的缺陷修复时递增修订号。版本号的含义清晰之后，仅看版本号就能了解每次发布的影响范围，依赖管理工具也才能据此判断升级是否安全。

2.2.3 为什么要学习 Git

从实用的角度出发，现代软件开发是多人、长期、迭代式的协作，代码的正确性依赖于对谁在何时为何修改了什么的完整记录。Git 正是记录这些历史的工具：它让每个人可以在自己的分支上独立开发而互不干扰，可以随时回退到任意历史版本，可以通过 diff 与 log 审查每一处改动。同时，如上面两个网页所述，良好的提交信息规范和版本号规范都建立在 Git 之上——commit 是 Change log 的来源，语义化版本描述的正是每次 Git tag 所代表的发布。不掌握 Git，就无法参与开源社区 (GitHub/GitLab 上的协作均以 Git 为基础)，也无法在团队中高效、安全地协作。

此外，从比较功利的角度来讲，若想在当代大型企业就业，似乎 Git 技能也是必不可少的。

因此学习 Git 是每个开发者必备的基本功。

2.3 分支管理与合并冲突处理

2.3.1 在 feature 与 main 分支上分别修改提交

先执行 `git branch feature` 新建 feature 分支，并执行 `git switch feature` 切换过去。在 feature 分支上将 main.c 中的 `printf` 修改为打印 "feature"。随后依次执行：

```
gcc main.c -o main.o
```

```
./main.o
git add .
git commit -m "feature task4"
git switch main
将 main.c 中同一位置的 printf 修改为打印 "main"后，执行：
gcc main.c -o main.o
./main.o
git add .
git commit -m "main task4"
```

2.3.2 合并并遇到冲突

在 main 分支上执行 `git merge feature`，Git 提示自动合并失败：main.c 中出现内容冲突（CONFLICT (content): Merge conflict in main.c），同时编译产物 main.o 作为二进制文件也无法自动合并。git status 显示两个文件处于 both modified 的未合并状态，Git 提示需要修复冲突后使用 `git commit` 完成合并，或者用 `git merge --abort` 放弃本次合并。如图 1 所示。

```
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab0/ics-lab$ git merge feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
warning: Cannot merge binary files: main.o (HEAD vs. feature)
Auto-merging main.o
CONFLICT (content): Merge conflict in main.o
Automatic merge failed; fix conflicts and then commit the result.
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab0/ics-lab$ git status
On branch main
Your branch is ahead of 'origin/main' by 2 commits.
(use "git push" to publish your local commits)

You have unmerged paths.
(fix conflicts and run "git commit")
(use "git merge --abort" to abort the merge)

Unmerged paths:
(use "git add <file>..." to mark resolution)
    both modified:   main.c
    both modified:   main.o
```

图 1 执行 `git merge feature` 后发生冲突

2.3.3 解决冲突

用 `cat main.c` 查看文件内容，可以看到 Git 在冲突位置插入了标记：“<<<<<<< HEAD”到“=====”之间是当前 main 分支的内容 `printf("main\n");`，“=====”到“>>>>>>> feature”之间是 feature 分支的内容 `printf("feature\n");`。

用 `vi main.c` 编辑文件，删除全部冲突标记并决定最终保留的内容。第一次编辑后仍残留了“>>>>>>> feature”标记，导致 `gcc` 编译报错“error: version

control conflict marker in file”；再次编辑彻底删除标记后，重新编译 gcc main.c -o main.o 并运行 ./main.o，程序正确输出 main，说明冲突已解决且代码可用。

随后执行 git add . 将解决后的文件加入暂存区，执行 git commit -m "conflict" 完成合并提交。此时再执行 git merge feature，Git 提示 “Already up to date.”，表明 feature 分支的内容已全部并入 main，合并与冲突解决流程完成。如图 2 所示。

```
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab0/ics-lab$ cat main.c
#include <stdio.h>

int main()
{
    // @TODO: print a sentence you want.
    <<<<<<< HEAD
    printf("main\n");
    =====
    printf("feature\n");
    >>>>>>> feature
}
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab0/ics-lab$ vi main.c
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab0/ics-lab$ gcc main.c -o main.o
main.c: In function 'main':
main.c:7:1: error: version control conflict marker in file
    7 | >>>>>>> feature
      | ^^^^^^^
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab0/ics-lab$ vi main.c
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab0/ics-lab$ gcc main.c -o main.o
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab0/ics-lab$ ./main.o
main
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab0/ics-lab$ git merge feature
error: Merging is not possible because you have unmerged files.
hint: Fix them up in the work tree, and then use 'git add/rm <file>'
hint: as appropriate to mark resolution and make a commit.
fatal: Exiting because of an unresolved conflict.
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab0/ics-lab$ git add .
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab0/ics-lab$ git commit -m "conflict"
[main 28cf5bf] conflict
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab0/ics-lab$ git merge feature
Already up to date.
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab0/ics-lab$ |
```

图 2 解决冲突后编译验证并提交，完成合并